

Aprendizado de Máquina B

Exercícios — Detecção de Objetos

Aluno: Lucas Affonso Pires | Matrícula: 2023028420

1 Algoritmos de Detecção de Objetos

Questão 1: Rótulo aproximado da imagem com o carro (imagem quadrada de lado 1).

Resposta:

O vetor de rótulo y para a imagem com o carro (classe $c = 2$) é:

$$y = [pc, bx, by, bh, bw, c1, c2, c3]$$
$$y \approx [1, 0.50, 0.60, 0.35, 0.50, 0, 1, 0]$$

Onde:

- $pc = 1$: há um objeto presente na imagem
- $bx \approx 0.50$, $by \approx 0.60$: coordenadas do centro do bounding box (centro horizontal, ligeiramente abaixo do meio vertical)
- $bh \approx 0.35$, $bw \approx 0.50$: altura e largura do bounding box em relação à imagem
- $c1 = 0$ (pedestre), $c2 = 1$ (carro), $c3 = 0$ (motocicleta)

Questão 2: Rótulo y para a imagem sem objetos de interesse (nenhuma das 3 classes presente).

Resposta:

Quando não há nenhum objeto das classes definidas na imagem, temos $pc = 0$. Os demais componentes são marcados como '?' (não importa), pois a função de perda ignora esses valores quando $pc = 0$:

$$y = [0, ?, ?, ?, ?, ?, ?, ?]$$

O único componente relevante é $pc = 0$ (nenhum objeto presente). Os valores bx , by , bh , bw e $c1$, $c2$, $c3$ não têm significado e são ignorados no cálculo da perda.

Questão 3: Fórmula da função de perda por soma de erros quadráticos para os dois casos.

Resposta:

Caso 1 — existe um objeto na imagem ($y1 = pc = 1$):

$$L(\hat{y}, y) = (\hat{y}_1 - y_1)^2 + (\hat{y}_2 - y_2)^2 + (\hat{y}_3 - y_3)^2 + (\hat{y}_4 - y_4)^2 \\ + (\hat{y}_5 - y_5)^2 + (\hat{y}_6 - y_6)^2 + (\hat{y}_7 - y_7)^2 + (\hat{y}_8 - y_8)^2$$

Ou seja, a perda é a soma dos erros quadráticos de todos os 8 componentes do vetor de saída (pc , bx , by , bh , bw , $c1$, $c2$, $c3$).

Caso 2 — não existe objeto na imagem ($y1 = pc = 0$):

$$L(\hat{y}, y) = (\hat{y}_1 - y_1)^2$$

A função de perda considera apenas o erro na predição de pc . Os demais componentes são ignorados, pois seus rótulos são '?' (não importam).

Questão 4: Conjunto de unidades de saída para detecção de lata de refrigerante.

Resposta:

Como há apenas uma classe (lata de refrigerante) e a bounding box é sempre quadrada e de tamanho fixo, a rede só precisa prever: (i) se há uma lata e (ii) a posição do seu centro. O vetor de saída é:

$$y = [pc, bx, by]$$

Onde:

- pc: probabilidade de haver uma lata — usa função de ativação Sigmoid, pois é uma classificação binária (0 ou 1)
- bx, by: coordenadas normalizadas do centro da lata — também com Sigmoid para manter os valores entre 0 e 1 (dentro dos limites da imagem)

Como bh e bw são constantes (tamanho fixo), não precisam ser previstos pela rede, simplificando o vetor de saída para dimensão 3.

Questão 5: Informações adicionais necessárias para treinamento supervisionado da rede de detecção.

Resposta:

Além das imagens e dos rótulos de classe dos objetos, é necessário fornecer:

- Bounding boxes anotadas: as coordenadas de cada objeto de interesse em cada imagem de treinamento, tipicamente representadas como (bx, by, bh, bw) — centro, altura e largura normalizados
- Identificação de células/regiões: em algoritmos como YOLO, qual célula do grid é responsável por cada objeto (baseado no centro do bounding box)

Em resumo: para cada imagem rotulada, é preciso informar, para cada objeto presente, sua classe e sua localização precisa (bounding box) na imagem.

2 Janelas Deslizantes

Questão 6: Forma fechada da saída de uma camada convolucional $2 \times 2 \times 2$ com 2 filtros 2×2 e sua equivalência com camada Fully Connected.

Resposta:

Para um volume de entrada W de dimensão $2 \times 2 \times 2$ (células $w_{\{ijc\}}$) e 2 filtros F de dimensão 2×2 (valores $F_{\{ijck\}}$, $k = 1, 2$), com stride 1 e sem padding, a saída tem dimensão $1 \times 1 \times 2$. A saída para cada filtro k é:

$$\begin{aligned} out_k &= g(\sum_i \sum_j \sum_c w_{\{ijc\}} \cdot F_{\{ijck\}}) \\ &= g(w_{111}F_{111k} + w_{121}F_{121k} + w_{211}F_{211k} + w_{221}F_{221k} \\ &\quad + w_{112}F_{112k} + w_{122}F_{122k} + w_{212}F_{212k} + w_{222}F_{222k}) \end{aligned}$$

Equivalência com Fully Connected:

- Nós de entrada: os 8 valores do volume de entrada ($2 \times 2 \times 2$), "desenrolados" em um vetor de dimensão 8
- Nós de saída: os 2 valores resultantes (um por filtro), vetor de dimensão 2
- Pesos: os coeficientes $F_{\{ijck\}}$ formam uma matriz de pesos de dimensão 8×2
- Viés: o bias b_k de cada filtro equivale ao bias da camada FC

A operação $z_k = \sum w_i \cdot F_{\{ik\}} + b_k$ é exatamente uma combinação linear, idêntica a uma camada FC. A função de ativação $g(\cdot)$ é aplicada da mesma forma em ambos os casos.

3 Detecção de Objetos / YOLO

Questão 7: Valor da IoU entre caixa 2×2 e caixa 2×3, com região sobreposta 1×1.

Resposta:

Calculando os componentes:

- Área da caixa 1 (2×2) = 4
- Área da caixa 2 (2×3) = 6
- Área de Interseção = 1×1 = 1
- Área de União = 4 + 6 - 1 = 9

$$\text{IoU} = \text{Interseção} / \text{União} = 1 / 9 \approx 0.111$$

Portanto, a IoU entre as duas caixas é aproximadamente 0.111 (11.1%).

Questão 8: Non-max suppression com limiar de descarte ≤ 0.4 e limiar de IoU = 0.5.

Resposta:

Da imagem, as caixas previstas são: tree 0.46, tree 0.74, car 0.62, motorcycle 0.58, car 0.73, pedestrian 0.98, car 0.26.

Passo 1 — Descartar caixas com probabilidade ≤ 0.4 :

- car 0.26 é descartada ($0.26 \leq 0.4$)

Restam: pedestrian 0.98, car 0.73, car 0.62, motorcycle 0.58, tree 0.74, tree 0.46

Passo 2 — Aplicar NMS por classe:

- Pedestres: só pedestrian 0.98 → mantida
- Carros: ordenar por probabilidade: car 0.73 e car 0.62. Se $\text{IoU} > 0.5$ (caixas sobrepostas), car 0.62 é suprimida → fica car 0.73
- Motocicletas: só motorcycle 0.58 → mantida
- Árvores: ordenar: tree 0.74 e tree 0.46. Se $\text{IoU} > 0.5$ (mesma região), tree 0.46 é suprimida → fica tree 0.74

Resultado final: 4 caixas permanecem → pedestrian 0.98, car 0.73, motorcycle 0.58, tree 0.74.

Questão 9: Repetir non-max suppression com limiar de descarte = 0.6.

Resposta:

Passo 1 — Descartar caixas com probabilidade ≤ 0.6 :

- tree 0.46 descartada
- motorcycle 0.58 descartada
- car 0.26 descartada

Restam: pedestrian 0.98, car 0.73, car 0.62, tree 0.74

Passo 2 — Aplicar NMS por classe:

- Pedestres: pedestrian 0.98 → mantida
- Carros: car 0.73 e car 0.62 → se $\text{IoU} > 0.5$, car 0.62 é suprimida → fica car 0.73

- Árvores: tree 0.74 → mantida (única restante)

Resultado final: 3 caixas permanecem → pedestrian 0.98, car 0.73, tree 0.74.

Questão 10: Cite 2 situações nas quais o algoritmo de non-max suppression é falho.

Resposta:

- Objetos diferentes muito próximos: quando dois objetos distintos da mesma classe estão sobrepostos ou adjacentes (ex: dois carros lado a lado), o NMS pode erroneamente suprimir a caixa de um dos objetos, pois a IoU entre as duas caixas será alta — fazendo com que apenas uma seja mantida.
 - Objeto grande cobrindo objeto pequeno (occlusão): quando um objeto menor está parcialmente dentro da bounding box de um objeto maior da mesma classe, o NMS tende a suprimir a caixa do objeto menor, perdendo a detecção. Ex: um carro ao fundo sendo suprimido pela detecção de um caminhão próximo.
-

Questão 11: No YOLO, como ocorre a detecção quando o objeto está em mais de uma célula do grid?

Resposta:

No YOLO, a responsabilidade pela detecção de um objeto é atribuída exclusivamente à célula do grid que contém o ponto central (centro geométrico) do bounding box do objeto. Mesmo que o objeto se estenda fisicamente por várias células, apenas a célula cujo centro de grade contém o midpoint do objeto é responsável por prever aquela detecção. As demais células não tentam detectar esse objeto. Isso simplifica o processo de treinamento, pois cada objeto tem exatamente uma célula responsável, eliminando ambiguidade nos rótulos.

Questão 12: Dimensão do volume de saída do YOLO com grade 19×19, 20 classes e 5 anchor boxes.

Resposta:

Para cada célula do grid e cada anchor box, a rede prevê:

- pc: probabilidade de objeto (1 valor)
- bx, by, bh, bw: coordenadas do bounding box (4 valores)
- c1, ..., c20: probabilidades de cada classe (20 valores)

Total por anchor box: $1 + 4 + 20 = 25$ valores

$$\text{Volume de saída} = 19 \times 19 \times (5 \times 25) = 19 \times 19 \times 125$$

Portanto, o volume de saída tem dimensões 19×19×125.

Questão 13: Por que o YOLO não é utilizado em todos os cenários, mesmo sendo mais rápido?

Resposta:

Apesar da velocidade, o YOLO apresenta as seguintes limitações:

- Menor acurácia geral: em benchmarks de precisão (mAP), versões mais antigas do YOLO ficavam abaixo das redes baseadas em regiões (R-CNN e variantes), especialmente em objetos pequenos

- Dificuldade com objetos pequenos: o YOLO processa a imagem em uma grade, e células pequenas têm campo receptivo limitado para capturar detalhes finos de objetos diminutos
 - Dificuldade com objetos próximos: cada célula pode prever apenas B objetos (anchor boxes), impossibilitando detectar múltiplos objetos com centros na mesma célula
 - Sensibilidade a aspect ratios não treinados: o modelo tem dificuldade em generalizar para bounding boxes de proporções muito diferentes das vistas no treinamento
 - Trade-off velocidade×precisão: em aplicações que exigem máxima acurácia (diagnóstico médico, inspeção industrial fina), a menor precisão é inaceitável mesmo com maior velocidade
-

Questão 14: Por que o YOLO não consegue detectar diferentes objetos muito próximos?

Resposta:

O YOLO divide a imagem em uma grade (ex: 19×19) e cada célula é responsável por detectar até B objetos (um por anchor box). O problema ocorre quando dois objetos distintos têm seus centros na mesma célula do grid — nesse caso, a rede tem apenas B "slots" para preencher naquela célula, e se $B < \text{número de objetos centrados nela}$, alguns objetos não serão detectados. Além disso, como cada célula prevê localização e classe independentemente, o modelo não tem mecanismo nativo para separar duas instâncias muito sobrepostas no mesmo receptive field.

Questão 15: Qual a vantagem de se utilizar múltiplas Anchor Boxes em detecção de objetos?

Resposta:

As principais vantagens são:

- Detecção de múltiplos objetos por célula: com múltiplas anchor boxes, uma célula do grid pode prever vários objetos simultaneamente (um por anchor box), superando a limitação de detecção única por célula
 - Especialização por forma/aspect ratio: cada anchor box é treinada para se especializar em objetos com formas diferentes (ex: anchor box vertical detecta melhor pedestres; anchor box horizontal detecta melhor carros), melhorando o ajuste dos bounding boxes
 - Melhor cobertura de escalas: permite ao modelo adaptar-se a objetos de diferentes tamanhos e proporções sem depender de múltiplas passagens na imagem
-

Questão 16: Cite duas formas de escolher Anchor Boxes e qual a vantagem de cada uma.

Resposta:

- Escolha manual baseada em conhecimento do domínio: define-se anchor boxes com formatos típicos dos objetos do domínio (ex: caixa alongada verticalmente para pedestres, quadrada para rostos, horizontal para carros). Vantagem: simples de implementar, não requer processamento adicional dos dados, e o especialista pode incorporar conhecimento prévio sobre os objetos.
 - K-means clustering sobre os bounding boxes do conjunto de treinamento: aplica-se o algoritmo k-means (usando IoU como métrica de distância) sobre todos os bounding boxes anotados no dataset. Os k centroides resultantes são usadas como anchor boxes. Vantagem: as anchor boxes são determinadas empiricamente a partir dos dados reais, representando melhor a distribuição das formas e tamanhos dos objetos do problema específico, tendendo a melhor desempenho.
-

4 Detecção de Objetos / R-CNN

Questão 17: Consequências de aumentar o stride em sliding windows e alternativas.

Resposta:

Ao aumentar o stride (passo) em uma implementação não-convolucional de janelas deslizantes:

- Custo computacional: reduz significativamente o número de janelas avaliadas, tornando o processo mais rápido
- Acurácia: diminui a acurácia de localização, pois o objeto pode não ser capturado perfeitamente por nenhuma das janelas (posições mais grosseiras), gerando bounding boxes imprecisas

Boa alternativa: implementação convolucional das janelas deslizantes (OverFeat / FCN). Em vez de executar a CNN em cada janela separadamente, converte-se as camadas FC em camadas convolucionais, permitindo processar a imagem inteira em uma única passagem e obter predições para todas as posições de forma compartilhada e muito mais eficiente.

Questão 18: Principal diferença entre R-CNN e Fast R-CNN.

Resposta:

R-CNN: extrai N regiões propostas da imagem (usando Selective Search), redimensiona cada região e executa a CNN independentemente em cada uma. Isso resulta em N passagens pela CNN, tornando o processo extremamente lento (30+ segundos por imagem).

Fast R-CNN: passa a imagem inteira pela CNN uma única vez, obtendo um mapa de features completo. Em seguida, projeta cada região proposta nesse mapa de features e extrai os vetores de features via RoI Pooling, sem reexecutar a CNN.

Vantagem: o Fast R-CNN é muito mais rápido (ordem de 25x) porque compartilha o custo computacional das convoluções entre todas as regiões propostas, em vez de recomputá-las para cada região individualmente.

Questão 19: Principal diferença entre Fast R-CNN e Faster R-CNN.

Resposta:

O gargalo de velocidade do Fast R-CNN é o algoritmo externo de proposição de regiões (Selective Search), que é lento e roda na CPU, tomando cerca de 2 segundos por imagem.

O Faster R-CNN introduz a Region Proposal Network (RPN), uma rede convolucional integrada que utiliza o mesmo mapa de features da CNN principal para propor regiões de interesse diretamente na GPU. Isso elimina o gargalo do Selective Search, tornando o sistema end-to-end treinável e aproximadamente 10x mais rápido que o Fast R-CNN.

Questão 20: Objetivo da camada de RoI Pooling e por que ela é necessária.

Resposta:

O objetivo da camada de RoI Pooling é converter features de regiões de interesse (RoIs) de tamanhos variáveis em um vetor de features de tamanho fixo.

Por que é necessária: os objetos em uma imagem têm tamanhos muito diferentes, gerando regiões propostas de dimensões variadas no mapa de features. As camadas Fully Connected subsequentes exigem uma entrada de tamanho fixo. A RoI Pooling resolve esse problema dividindo cada RoI em uma grade $H \times W$ de sub-regiões e aplicando max pooling em cada sub-região, produzindo sempre um mapa de saída $H \times W$ independente do tamanho da RoI original. Sem essa camada, seria necessário redimensionar cada região separadamente antes de classificá-la — processo ineficiente que o Fast R-CNN e o Faster R-CNN eliminam ao compartilhar o mapa de features.